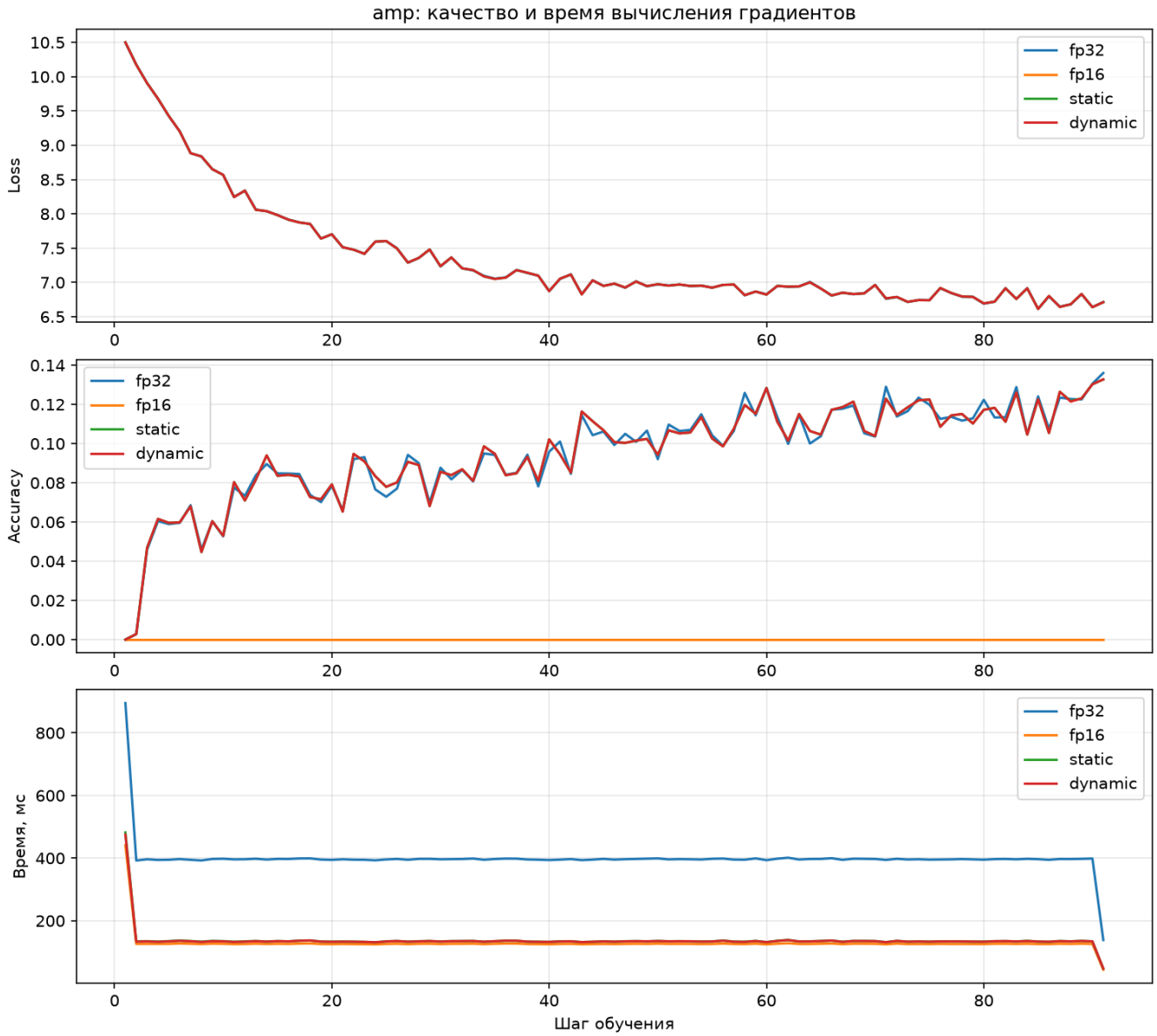
Видеокарта: RTX 5070.

AMP

Конфигурация: data/test-00000-of-00001.txt, загрузчик base, batch size 32, 1 эпоха, seed 42. Сравнивались fp32, fp16, static и dynamic; для scaling начальное значение - 65536. Для dynamic: factor 2, patience 80, min scale 1, max scale 16777216.

Режим	Среднее время подсчёта градиента на шаг	Относительно FP32	Средний loss	Средняя accuracy
FP32	399,1 мс	-	7,355	9,54%
FP16 без scaling	128,5 мс	-67,8%	NaN	0%
Static scaling	137,1 мс	-65,6%; в 2,91 раза быстрее	7,355	9,54%
Dynamic scaling	136,8 мс	-65,7%; в 2,92 раза быстрее	7,355	9,54%

FP16 без scaling разошёлся: loss стал NaN, accuracy упала до нуля. Static и dynamic сохранили показатели качества на уровне FP32 и ускорили подсчёт градиента.



Loading

Конфигурация: data/test-00000-of-00001.txt, fp32, batch size 16, 1 эпоха, seed 42. Сравнивались загрузчики base, standard, balanced (k=20) и sequenced.

Загрузчик	Время обучения	Относительно base	Шагов	Пик allocated / reserved
Base	37,99 с	-	181	4,74 / 5,58 ГиБ
Standard	38,23 с	на 0,6% медленнее	181	4,73 / 21,54 ГиБ
Balanced	12,30 с	на 67,6% быстрее	193	3,92 / 10,88 ГиБ
Sequenced	13,35 с	на 64,9% быстрее	477	1,22 / 1,41 ГиБ

Standard не обогнал base: **38,23 против 37,99 с**. При почти одинаковом пике выделенной памяти PyTorch зарезервировал **21,54 против 5,58 ГиБ**. Вероятное объяснение - разные формы тензоров: standard дополняет последовательности до максимальной длины **в каждом отдельном батче**, поэтому размеры меняются между шагами.

Balanced показал наименьшее время обучения, а Sequenced - наименьший пик памяти.

Profiling

Конфигурация: data/test-00000-of-00001.txt, загрузчик sequenced, dynamic, batch size 48, 1 эпоха, seed 42; scale 65536, factor 2, patience 80, min scale 1, max scale 16777216. В каждом запуске выполнено 465 шагов.

Правка	Время обучения	Относительно исходного	Изменение профиля
Без изменений	11,181 с	-	-
set_to_none=True	10,594 с	-5,2%	aten::zero_: 245 до 84
Удаление .item() из вывода каждого шага	10,832 с	-3,1%	aten::item: 497 до 483; синхронизации: 238 до 224
TransformerDecoderLayer заменён на TransformerEncoderLayer	9,792 с	-12,4%	вызовы attention: 14 до 7
Удаление вычисления неиспользуемого y	11,088 с	-0,8%	aten::addmm: 70 до 63
hidden_dim: 1020 до 1024	10,841 с	-3,0%	math attention сменился на efficient_attention
Все правки вместе	8,847 с	-20,9%; в 1,26 раза быстрее	attention: 14 до 7; zero_: 245 до 77; синхронизации: 238 до 182

set_to_none=True. Вместо записи нулей в существующие тензоры градиенты сбрасываются в None. Число операций обнуления уменьшается.

Удаление .item(). Вызов .item() при выводе каждого шага синхронизирует CPU и GPU. После удаления таких вызовов синхронизаций стало меньше.

Замена слоя. TransformerDecoderLayer считает self-attention и дополнительный attention к memory. В исходном коде в оба входа передавался x. После замены на TransformerEncoderLayer остаётся один attention; число вызовов сократилось с 14 до 7.

Удаление y. Вычислялся тензор y, который далее не использовался в обучении. Его удаление убрало лишнюю операцию.

hidden_dim. Ширина изменена с 1020 до 1024 для выравнивания по степени двойки. В профиле вместо math attention стал вызываться efficient_attention.

После всех правок пиковая выделенная GPU-память снизилась с **1,250 до 1,098 ГиБ**, зарезервированная - с **1,439 до 1,148 ГиБ**.